

Instruction for Modules Build in Visual Studio

Already assembled programs (modules) for Windows are in folder '**Folder_for_Calculations**' in https://github.com/Persova-LMDPHT/FDEM_BfiPF.

To rebuild all programs (modules), download the contents of the '**CODE**' folder from https://github.com/Persova-LMDPHT/FDEM_BfiPF. The 'CODE' folder contains the source code for all modules (each module is located in its own folder inside the 'CODE' folder, with the same folder name as the module).

To build programs (modules) in Microsoft Visual Studio (please see section 'Requirements' in the file 'readme'), you should create projects for each module:

1	Module ' av ' (program language: C++; Intel MKL is used).....	2
2	Module ' bound ' (program language: C++).....	2
3	Module ' CalcFreq ' (program language: C++).....	3
4	Module ' CalcHarm2DHEL ' (program language: C++).....	3
5	Module ' CalcHarm2DHEL_AV ' (program language: C++).....	3
6	Module ' CalcHarm2DHEL_U ' (program language: C++).....	4
7	Module ' CalcHarm2D_Ax ' (program language: C++; Intel MKL is used).....	4
8	Module ' CalcHarm3D ' (program language: C++; Intel MKL and OpenMP are used).....	5
9	Module ' Mesh2D_FD ' (program language: Fortran).....	6
10	Module ' OutputSmooth2DHarm_Ax ' (program language: C++; OpenMP is used).....	6
11	Module ' OutputSmooth2DHarm_U ' (program language: C++; OpenMP is used).....	7
12	Module ' OutputSmoothAV2DHarm_Ax ' (program language: C++).....	7
13	Module ' OutputSmoothAV2DHarm_Er ' (program language: C++).....	8
14	Module ' OutputSmoothAV2DHarm_Ez ' (program language: C++).....	8
15	Module ' RegularMeshBuilder ' (program language: C++).....	9
16	Module ' SumHarm2D3D ' (program language: C++).....	9
17	Module ' u ' (program language: C++; Intel MKL is used).....	10
18	Module ' UnloadAnomalHarm ' (program language: C++).....	10
19	Module ' CalcStarter ' (program language: C++).....	11

After the modules build, put CalcStarter.exe file to your folder 'Folder_for_Calculations' and other 18 executable (*.exe) files to the folder 'Modules' in your 'Folder_for_Calculations'.

When running programs (modules), DLL files may be required, which must be placed to the 'Modules' folder. For user convenience, we have placed the main DLL files in the '**Folder_for_Calculations/Modules**' folder in https://github.com/Persova-LMDPHT/FDEM_BfiPF.

Your 'Folder_for_Calculations' is ready to use!

To perform the calculation, add the Input files (they are described in the 'readme' file and in the user_manual.txt) to your 'Folder_for_Calculations' and run CalcStarter.exe.

1 Module '**av**' (program language: C++; Intel MKL is used)

1.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**av**'

1.2 Copy all files from folder '**av**' in folder 'CODE' to the folder with your project '**av**'

1.3 Adding *.cpp files to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.

- Navigate to the location of *.cpp files in folder with code of module '**av**'. Select all *.cpp files to add.

1.4 Adding *.h files to Visual Studio project:

- Right-click on the 'Header Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.

- Navigate to the location of *.h files in folder with code of module '**av**'. Select all *.h files to add.

1.5 To use Intel MKL in Visual Studio (install Intel oneAPI Base Toolkit which include the oneMKL libraries and Visual Studio integration):

- Right-click the project in the Solution Explorer and select Properties.

- Navigate to Configuration Properties > Intel Performance Libraries.

- Select the Use oneMKL property and change the setting to Parallel.

- Click Apply and OK

1.6 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.

- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.

- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.

- In the main menu, go to Build > Build Solution

1.7 The executable (**av.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **av.exe** file and copy it to folder 'Modules' in your folder for calculations.

2 Module '**bound**' (program language: C++)

2.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**bound**'

2.2 Copy all files from folder '**bound**' in folder 'CODE' to the folder with your project '**bound**'

2.3 Adding *.cpp files to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.

- Navigate to the location of *.cpp files in folder with code of module '**bound**'. Select all *.cpp files to add.

2.4 Adding *.h files to Visual Studio project:

- Right-click on the 'Header Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.

- Navigate to the location of *.h files in folder with code of module '**bound**'. Select all *.h files to add.

2.5 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.

- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.

- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.

- In the main menu, go to Build > Build Solution

2.6 The executable (**bound.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **bound.exe** file and copy it to folder 'Modules' in your folder for calculations.

3 Module '**CalcFreq**' (program language: C++)

3.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**CalcFreq**'

3.2 Copy main.cpp from folder '**CalcFreq**' in folder 'CODE' to the folder with your project '**CalcFreq**'

3.3 Adding main.cpp to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of main.cpp in folder with code of module '**CalcFreq**'. Select main.cpp to add.

3.4 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

3.5 The executable (**CalcFreq.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **CalcFreq.exe** file and copy it to folder 'Modules' in your folder for calculations.

4 Module '**CalcHarm2DHEL**' (program language: C++)

4.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**CalcHarm2DHEL**'

4.2 Copy main.cpp from folder '**CalcHarm2DHEL**' in folder 'CODE' to the folder with your project '**CalcHarm2DHEL**'

4.3 Adding main.cpp to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of main.cpp in folder with code of module '**CalcHarm2DHEL**'. Select main.cpp to add.

4.4 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

4.5 The executable (**CalcHarm2DHEL.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **CalcHarm2DHEL.exe** file and copy it to folder 'Modules' in your folder for calculations.

5 Module '**CalcHarm2DHEL_AV**' (program language: C++)

5.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**CalcHarm2DHEL_AV**'

5.2 Copy main.cpp from folder '**CalcHarm2DHEL_AV**' in folder 'CODE' to the folder with your project '**CalcHarm2DHEL_AV**'

5.3 Adding main.cpp to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of main.cpp in folder with code of module '**CalcHarm2DHEL_AV**'. Select main.cpp to add.

5.4 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

5.5 The executable (**CalcHarm2DHEL_AV.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **CalcHarm2DHEL_AV.exe** file and copy it to folder 'Modules' in your folder for calculations.

6 Module '**CalcHarm2DHEL_U**' (program language: C++)

6.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**CalcHarm2DHEL_U**'

6.2 Copy main.cpp from folder '**CalcHarm2DHEL_U**' in folder 'CODE' to the folder with your project '**CalcHarm2DHEL_U**'

6.3 Adding main.cpp to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of main.cpp in folder with code of module '**CalcHarm2DHEL_U**'. Select main.cpp to add.

6.4 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

6.5 The executable (**CalcHarm2DHEL_U.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **CalcHarm2DHEL_U.exe** file and copy it to folder 'Modules' in your folder for calculations.

7 Module '**CalcHarm2D_Ax**' (program language: C++; Intel MKL is used)

7.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**CalcHarm2D_Ax**'

7.2 Copy all files from folder '**CalcHarm2D_Ax**' in folder 'CODE' to the folder with your project '**CalcHarm2D_Ax**'

7.3 Adding *.cpp files to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.cpp files in folder with code of module '**CalcHarm2D_Ax**'. Select all *.cpp files to add.

7.4 Adding *.h files to Visual Studio project:

- Right-click on the 'Header Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.h files in folder with code of module '**CalcHarm2D_Ax**'. Select all *.h files to add.

7.5 To use Intel MKL in Visual Studio (install Intel oneAPI Base Toolkit which include the oneMKL libraries and Visual Studio integration):

- Right-click the project in the Solution Explorer and select Properties.
- Navigate to Configuration Properties > Intel Performance Libraries.
- Select the Use oneMKL property and change the setting to Parallel.
- Click Apply and OK

7.6 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

7.7 The executable (**CalcHarm2D_Ax.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **CalcHarm2D_Ax.exe** file and copy it to folder 'Modules' in your folder for calculations.

8 Module '**CalcHarm3D**' (program language: C++; Intel MKL and OpenMP are used)

8.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**CalcHarm3D**'

8.2 Copy all files from folder '**CalcHarm3D**' in folder 'CODE' to the folder with your project '**CalcHarm3D**'

8.3 Adding *.cpp files to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.cpp files in folder with code of module '**CalcHarm3D**'. Select all *.cpp files to add.

8.4 Adding *.h files to Visual Studio project:

- Right-click on the 'Header Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.h files in folder with code of module '**CalcHarm3D**'. Select all *.h files to add.

8.5 To use Intel MKL in Visual Studio (install Intel oneAPI Base Toolkit which include the oneMKL libraries and Visual Studio integration):

- Right-click the project in the Solution Explorer and select Properties.
- Navigate to Configuration Properties > Intel Performance Libraries.
- Select the Use oneMKL property and change the setting to Parallel.
- Click Apply and OK

8.6 To use OpenMP in Visual Studio Project:

- Right-click the project in the Solution Explorer and select Properties.
- Navigate to Configuration Properties > C/C++ > Language.
- Find the OpenMP Support setting.
- Change the dropdown to Yes (/openmp)
- Click Apply and OK

8.7 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

8.8 The executable (**CalcHarm3D.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **CalcHarm3D.exe** file and copy it to folder 'Modules' in your folder for calculations.

9 Module '**Mesh2D_FD**' (program language: Fortran)

9.1 Create a new project in Microsoft Visual Studio (Empty Project for Fortran) with name '**Mesh2D_FD**'

9.2 Copy all files from folder '**Mesh2D_FD**' in folder 'CODE' to the folder with your project '**Mesh2D_FD**'

9.3 Adding *.for files to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.for files in folder with code of module '**Mesh2D_FD**'. Select all *.for files to add.

9.4 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

9.5 The executable (**Mesh2D_FD.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **Mesh2D_FD.exe** file and copy it to folder 'Modules' in your folder for calculations.

10 Module '**OutputSmooth2DHarm_Ax**' (program language: C++; OpenMP is used)

10.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**OutputSmooth2DHarm_Ax**'

10.2 Copy all files from folder '**OutputSmooth2DHarm_Ax**' in folder 'CODE' to the folder with your project '**OutputSmooth2DHarm_Ax**'

10.3 Adding *.cpp files to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.cpp files in folder with code of module '**OutputSmooth2DHarm_Ax**'. Select all *.cpp files to add.

10.4 Adding *.h files to Visual Studio project:

- Right-click on the 'Header Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.h files in folder with code of module '**OutputSmooth2DHarm_Ax**'. Select all *.h files to add.

10.5 To use OpenMP Support in Visual Studio Project:

- Right-click the project in the Solution Explorer and select Properties.
- Navigate to Configuration Properties > C/C++ > Language.
- Find the OpenMP Support setting.
- Change the dropdown to Yes (/openmp)
- Click Apply and OK

10.6 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.

- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution
 - 10.7 The executable (**OutputSmooth2DHarm_Ax.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **OutputSmooth2DHarm_Ax.exe** file and copy it to folder 'Modules' in your folder for calculations.

11 Module '**OutputSmooth2DHarm_U**' (program language: C++; OpenMP is used)

- 11.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**OutputSmooth2DHarm_U**'
- 11.2 Copy all files from folder '**OutputSmooth2DHarm_U**' in folder 'CODE' to the folder with your project '**OutputSmooth2DHarm_U**'
- 11.3 Adding *.cpp files to Visual Studio project:
 - Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
 - Navigate to the location of *.cpp files in folder with code of module '**OutputSmooth2DHarm_U**'. Select all *.cpp files to add.
- 11.4 Adding *.h files to Visual Studio project:
 - Right-click on the 'Header Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
 - Navigate to the location of *.h files in folder with code of module '**OutputSmooth2DHarm_U**'. Select all *.h files to add.
- 11.5 To use OpenMP Support in Visual Studio Project:
 - Right-click the project in the Solution Explorer and select Properties.
 - Navigate to Configuration Properties > C/C++ > Language.
 - Find the OpenMP Support setting.
 - Change the dropdown to Yes (/openmp)
 - Click Apply and OK
- 11.6 To Build a 64-bit Release in Visual Studio:
 - Go to the main menu and select Build > Configuration Manager.
 - In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
 - In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
 - In the main menu, go to Build > Build Solution
 - 11.7 The executable (**OutputSmooth2DHarm_U.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **OutputSmooth2DHarm_U.exe** file and copy it to folder 'Modules' in your folder for calculations.

12 Module '**OutputSmoothAV2DHarm_Ax**' (program language: C++)

- 12.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**OutputSmoothAV2DHarm_Ax**'
- 12.2 Copy all files from folder '**OutputSmoothAV2DHarm_Ax**' in folder 'CODE' to the folder with your project '**OutputSmoothAV2DHarm_Ax**'
- 12.3 Adding *.cpp files to Visual Studio project:
 - Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.

- Navigate to the location of *.cpp files in folder with code of module '**OutputSmoothAV2DHarm_Ax**'. Select all *.cpp files to add.

12.4 Adding *.h files to Visual Studio project:

- Right-click on the 'Header Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.

- Navigate to the location of *.h files in folder with code of module '**OutputSmoothAV2DHarm_Ax**'. Select all *.h files to add.

12.5 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.

- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.

- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.

- In the main menu, go to Build > Build Solution

12.6 The executable (**OutputSmoothAV2DHarm_Ax.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **OutputSmoothAV2DHarm_Ax.exe** file and copy it to folder 'Modules' in your folder for calculations.

13 Module '**OutputSmoothAV2DHarm_Er**' (program language: C++)

13.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**OutputSmoothAV2DHarm_Er**'

13.2 Copy all files from folder '**OutputSmoothAV2DHarm_Er**' in folder 'CODE' to the folder with your project '**OutputSmoothAV2DHarm_Er**'

13.3 Adding *.cpp files to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.

- Navigate to the location of *.cpp files in folder with code of module '**OutputSmoothAV2DHarm_Er**'. Select all *.cpp files to add.

13.4 Adding *.h files to Visual Studio project:

- Right-click on the 'Header Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.

- Navigate to the location of *.h files in folder with code of module '**OutputSmoothAV2DHarm_Er**'. Select all *.h files to add.

13.5 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.

- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.

- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.

- In the main menu, go to Build > Build Solution

13.6 The executable (**OutputSmoothAV2DHarm_Er.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **OutputSmoothAV2DHarm_Er.exe** file and copy it to folder 'Modules' in your folder for calculations.

14 Module '**OutputSmoothAV2DHarm_Ez**' (program language: C++)

14.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**OutputSmoothAV2DHarm_Ez**'

14.2 Copy all files from folder '**OutputSmoothAV2DHarm_Ez**' in folder 'CODE' to the folder with your project '**OutputSmoothAV2DHarm_Ez**'

14.3 Adding *.cpp files to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.cpp files in folder with code of module '**OutputSmoothAV2DHarm_Ez**'. Select all *.cpp files to add.

14.4 Adding *.h files to Visual Studio project:

- Right-click on the 'Header Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.h files in folder with code of module '**OutputSmoothAV2DHarm_Ez**'. Select all *.h files to add.

14.5 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

14.6 The executable (**OutputSmoothAV2DHarm_Ez.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **OutputSmoothAV2DHarm_Ez.exe** file and copy it to folder 'Modules' in your folder for calculations.

15 Module '**RegularMeshBuilder**' (program language: C++)

15.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**RegularMeshBuilder**'

15.2 Copy all files from folder '**RegularMeshBuilder**' in folder 'CODE' to the folder with your project '**RegularMeshBuilder**'

15.3 Adding *.cpp files to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.cpp files in folder with code of module '**RegularMeshBuilder**'. Select all *.cpp files to add.

15.4 Adding *.h files to Visual Studio project:

- Right-click on the 'Header Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.h files in folder with code of module '**RegularMeshBuilder**'. Select all *.h files to add.

15.5 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

15.6 The executable (**RegularMeshBuilder.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **RegularMeshBuilder.exe** file and copy it to folder 'Modules' in your folder for calculations.

16 Module '**SumHarm2D3D**' (program language: C++)

16.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**SumHarm2D3D**'

16.2 Copy main.cpp from folder '**SumHarm2D3D**' in folder 'CODE' to the folder with your project '**SumHarm2D3D**'

16.3 Adding main.cpp to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of main.cpp in folder with code of module '**SumHarm2D3D**'. Select main.cpp to add.

16.4 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

16.5 The executable (**SumHarm2D3D.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **SumHarm2D3D.exe** file and copy it to folder 'Modules' in your folder for calculations.

17 Module '**u**' (program language: C++; Intel MKL is used)

17.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**u**'

17.2 Copy all files from folder '**u**' in folder 'CODE' to the folder with your project '**u**'

17.3 Adding *.cpp files to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.cpp files in folder with code of module '**u**'. Select all *.cpp files to add.

17.4 Adding *.h files to Visual Studio project:

- Right-click on the 'Header Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.h files in folder with code of module '**u**'. Select all *.h files to add.

17.5 To use Intel MKL in Visual Studio (install Intel oneAPI Base Toolkit which include the oneMKL libraries and Visual Studio integration):

- Right-click the project in the Solution Explorer and select Properties.
- Navigate to Configuration Properties > Intel Performance Libraries.
- Select the Use oneMKL property and change the setting to Parallel.
- Click Apply and OK

17.6 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

17.7 The executable (**u.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **u.exe** file and copy it to folder 'Modules' in your folder for calculations.

18 Module '**UnloadAnomalHarm**' (program language: C++)

18.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**UnloadAnomalHarm**'

18.2 Copy all files from folder '**UnloadAnomalHarm**' in folder 'CODE' to the folder with your project '**UnloadAnomalHarm**'

18.3 Adding *.cpp files to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.cpp files in folder with code of module '**UnloadAnomalHarm**'. Select all *.cpp files to add.

18.4 Adding *.h files to Visual Studio project:

- Right-click on the 'Header Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of *.h files in folder with code of module '**UnloadAnomalHarm**'. Select all *.h files to add.

18.5 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

18.6 The executable (**UnloadAnomalHarm.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **UnloadAnomalHarm.exe** file and copy it to folder 'Modules' in your folder for calculations.

19 Module '**CalcStarter**' (program language: C++)

19.1 Create a new project in Microsoft Visual Studio (Empty Project for C++) with name '**CalcStarter**'

19.2 Copy main.cpp from folder '**CalcStarter**' in folder 'CODE' to the folder with your project '**CalcStarter**'

19.3 Adding main.cpp to Visual Studio project:

- Right-click on the 'Source Files' in the 'Solution Explorer' panel and select Add > Existing Item. A file explorer dialog box will open.
- Navigate to the location of main.cpp in folder with code of module '**CalcStarter**'. Select main.cpp to add.

19.4 To Build a 64-bit Release in Visual Studio:

- Go to the main menu and select Build > Configuration Manager.
- In the Configuration Manager dialog box, find the Active solution platform dropdown list. Select x64 from the list.
- In the same Configuration Manager dialog box, find the Active solution configuration dropdown list. Select "Release" from the list.
- In the main menu, go to Build > Build Solution

19.5 The executable (**CalcStarter.exe**) file is typically located in the Release subdirectory within the main project folder, after the project has been successfully built. Find **CalcStarter.exe** file and copy it to folder 'Modules' in your folder for calculations.